PL/SQL to proceduralny język programowania dla relacyjnych baz danych

Kod + SQL = PL/SQL

PL/SQL charakteryzuje się strukturą blokową

- sekcja deklaracji (opcjonalna) - DECLARE
- sekcja wywoływania EXECUTABLE (wymagana) - BEGIN … END
- sekcja wyjątków EXCEPTION HANDLING (opcjonalna) - EXCEPTION

ROZDZIAŁ 2

SŁOWNIK

Parametr - wartość przekazana to programu przez użytkownika lub inny program,
Zmienna - używana do tymczasowego przechowywania i przetwarzania danych,
NCLOB - National Language Character Large Object - przechowuje ogromne bloki NCHAR danych w bazie,
Object - obiekt schematu z nazwą, atrybutami i metodami
Reference - przechowuje wartości zwane wskaźniki, które wskazują na lokalizację w pamięci,
Scalar - przechowuje pojedynczą wartość bez wewnętrznych komponentów,
BFILE - przechowuje duże obiekty binarne poza bazą danych,
BLOB - przechowuje duże obiekty binarne w bazie danych,
CLOB - przechowuje duże bloki danych znakowych w bazie danych,
Composite - zawiera elementy wewnętrzne, które mogą być przetwarzane osobno,
LOB - typ danych przechowujący wartości zwane lokalizatorami, które które określają lokalizację dużych obiektów (takich jak obrazy graficzne),
%TYPE - Atrybut służący do deklarowania zmiennej na podstawie innej, wcześniej zadeklarowanej zmiennej lub kolumny tabeli.
BOOLEAN - typ danych, które może zawierać jedynie wartości - TRUE, FALSE lub NULL
Konwersja jawna - konwertuje wartości z jednego typu danych na inny przy użyciu wbudowanych funkcji.
Konwersja niejawna - konwertuje dynamicznie typy danych, jeśli są one wymieszane w poleceniu
Etykieta bloku – nazwa nadana blokowi kodu, która umożliwia odwoływanie się do zmiennych i ich wartości w tym bloku kodu z innych bloków kodu.
Zakres zmiennej – składa się ze wszystkich bloków, w których zmienna jest lokalna (blok deklarujący) lub globalna (zagnieżdżone bloki w bloku deklarującym)
Widoczność zmiennej – część programu, w której można uzyskać dostęp do zmiennej bez użycia kwalifikatora

DEKLARACJE ZMIENNYCH

w sekcji DECLARE, np.

```
DECLARE
        v_counter INTEGER := 10;
```

Można też zadeklarować zmienną bez wartości i później w sekcji BEGIN to zmienić, np.

```
DECLARE
        v_name VARCHAR2(20);
BEGIN
        v_name := 'Kamil';
```

można też użyć funkcji (parametru) i przekazać do zmiennej wartość tego parametru w sekcji executable, np.

```
CREATE FUNCTION num_characters (p_string IN VARCHAR2)
RETURN INTEGER IS
        v_num_characters INTEGER;
BEGIN
        SELECT LENGTH(p_string) INTO v_num_characters FROM DUAL;
```

teraz wystarczy zadeklarować pod zmienną jakiegoś stringa

JEDNOSTKI LEKSYKALNE

- **Identyfikatory -** to nazwy obiektów, zmiennych, funkcji i procedur
- **zarezerwowane słowa -** typu BEGIN, END, DECLARE, SELECT itp.
- **ograniczniki (delimiters) -** (+, ;, :=, itp.)
- **literały -** Literał to wyraźna wartość liczbowa, ciąg znaków, data lub wartość logiczna (np. imię osoby
  może być literałem przechowywanym w zmiennej).
- **komentarze** - informują nas co kod ma na celu wykonać i są ignorowane przez kompilator

SCALAR DATA TYPES:

- Character
- Number
- Date
- Boolean

SCALAR CHARACTER data types:

- CHAR
- VARCHAR2
- LONG

SCALAR NUMERIC data types:

- NUMBER
- PLS_INTEGER

SCALAR DATE data types:

- DATE
- TIMESTAMP
- TIMESTAMP WITH TIME ZONE

UŁATWIENIE DO TYPÓW ZMIENNYCH W PL/SQL
jeśli nie chcesz mieć problemów z tytułu różnych typów (np. kolumna w tabeli ma typ VARCHAR2, a ty stworzysz zmienną typu NUMBER i będziesz chciał umieścić dane z tabeli do tej zmiennej to napotkasz niezgodność typów). W tym celu należy użyć bezpośredniego odnośnika do typu zmiennej przechowywanej w kolumnie, np.

v_emp_name employees.first_name%TYPE;

można też całą kolumnę na raz

v_emp_record employees%ROWTYPE;
LOB (Large Object) Data Types:

- Character large object (CLOB),
- Binary large object (BLOB),
- Binary file (BFILE)
- National language character large object (NCLOB)

Przykład deklaracji zmiennych

```
DECLARE
        v_first_name employees.first_name%TYPE;
        v_salary employess.salary%TYPE;
        v_old_salary v_salary%TYPE;
        v_new_salary v_salary%TYPE;
```

%TYPE pomaga:

- uniknąć błędu związanego z niepoprawnym typem danych,
- nie trzeba zmieniać deklaracji zmiennej, w przypadku gdy zmieni się coś w głównej tabeli (na przykład zmiana typu kolumny)

KONWERSJA NIEJAWNA

Może nastąpić między:

- DATE -> LONG, VARCHAR2
- LONG -> VARCHAR2
- NUMBER -> LONG, PLS_INTEGER, VARHCAR2
- PLS_INTEGER -> LONG, NUMBER, VARCHAR2
- VARCHAR2 -> DATE, LONG, NUMBER, PLS_INTEGER

Przykład niejawnej konwersji:

```
DECLARE
        v_salary NUMBER(6) := 6000;
        v_sal_increaseVARCHAR2(5) := '1000';
        v_total_salaryv_salary%TYPE;
BEGIN
        v_total_salary:= v_salary+ v_sal_increase;
        DBMS_OUTPUT.PUT_LINE(v_total_salary); END;
```

MINUSY NIEJAWNEJ KONWERSJI:

- może być wolniejsza
- tracisz kontrolę nad programem, ponieważ oddajesz to w pełni temu jak ORACLE traktuje dane
- gdy dojdzie do konwersji, kod może zostać naruszony
- kod który używa tej konwersji jest trudniejszy do zrozumienia

KONWERSJA JAWNA

- TO_CHAR()
- TO_DATE()
- TO_NUMBER()

PRZYKŁAD ZAGNIEŻDŻONEGO BLOKU

```
DECLARE
        v_outer_variableVARCHAR2(20) :='GLOBAL VARIABLE';
BEGIN
        DECLARE
                v_inner_variable VARCHAR2(20):= 'LOCAL VARIABLE';
        BEGIN
                DBMS_OUTPUT.PUT_LINE(v_inner_variable);
                DBMS_OUTPUT.PUT_LINE(v_outer_variable);
        END;
        DBMS_OUTPUT.PUT_LINE(v_outer_variable); END;
```

Wniosek z tego nasuwa się: zmienną wewnętrzną możesz wywołać jedynie w funkcji wewnętrznej, natomiast zewnętrzną w obydwu miejscach

DOBRE PRAKTYKI PROGRAMISTYCZNE:

- konwersja jawna > konwersja niejawna
- odpowiednie nazwy zmiennych (żeby było wiadomo ocb)
- jedna deklaracja zmiennych na linię (większa czytelność)
- unikanie niejednoznaczności przy wyborze identyfikatorów
- używanie %TYPE, zamiast ręcznie
- NOT NULL dla zmiennej, która musi coś mieć w środku na pewno!

WYTYCZNE DOTYCZĄCE PROGRAMOWANIA:

- dokumentacja kodu przy pomocy komentarzy
- opracowanie konwencji przypadku dla kodu
- tak samo jak nazw zmiennych
- tworzenie tabulatorów dla czytelności

ROZDZIAŁ 3

SŁOWNIK:

DATA DEFINITION LANGUAGE (DDL) - kiedy tworzysz, zmieniasz lub usuwasz obiekt w bazie danych
DATA MANIPULATION LANGUAGE (DML) - kiedy zmieniasz dane w obiekcie (gdy np. wstawiasz lub usuwasz wiersze)
DELETE - Polecenie służące do usuwania istniejących wierszy w kolumn w tabeli
INSERT - polecenie służące do dodawania nowych wierszy do tabeli
MERGE - polecenie służące do wstawiania lub aktualizacji tabeli, na podstawie pasujących do siebie danych w tabeli źródłowej
UPDATE - polecenie służące do modyfikacji istniejących wierszy w tabeli
COMMIT - polecenie, które potwierdza permanentnie zmiany w bazie danych
END - słowo kluczowe, które informuje blok PL/SQL o końcu bloku kodu
ROLLBACK - używany do wycofywania zmian, które zostały dokonane na bazie przed **ostatnim** commitem
SAVEPOINT - używany do zaznaczenia jakiegoś momentu podczas przetwarzania transakcji
Transaction - nierozłączna lista operacji, która musi zostać wykonana w całości, żeby była potwierdzona

INSERT wymaga:

- nazwy tabeli
- wartości dla każdej kolumny w tabeli

można to zrobić jawnie lub niejawnie, (jawnie gdy podajesz nazwę każdej kolumny, niejawnie gdy podajesz nazwę całej tabeli + wartości jakie chcesz w niej umieścić)

UPDATE:

- za pomocą SET możesz zmienić wartość pojedynczej kolumny
- klauzula WHERE pozwala na to, że nie zmienisz wszystkiego, tylko określone wiersze

DELETE:

- tabela musi istnieć
- musisz podać nazwę tej tabeli
- i opcjonalnie klauzulę WHERE

MERGE:

- inną tabelę wciskasz w tabelę źródłową, przykładowo tworzysz tabele:

```
CREATE TABLE bonuses
        (employee_id NUMBER(6,0) NOT NULL,
        bonus NUMBER(8,2) DEFAULT 0);
```

I chcesz do niej dodać dane z tabeli employees:

```
 INSERT INTO bonuses(employee_id)
        (SELECT employee_id
        FROM employees
        WHERE salary < 10000);
```

DDL - CREATE TABLE, ALTER TABLE, DROP TABLE
DCL - GRANT, REVOKE

KURSORY

Za każdym razem, gdy instrukcja SQL ma zostać wykonana,
serwer Oracle przydziela prywatny obszar pamięci do
przechowywania instrukcji SQL i używanych przez nią danych.

Ten obszar pamięci nazywany jest kursorem niejawnym.
Ponieważ ten obszar pamięci jest automatycznie zarządzany
przez serwer Oracle, nie masz nad nim bezpośredniej kontroli.

Możesz jednak użyć predefiniowanych zmiennych PL/SQL,
nazywanych atrybutami kursora niejawnego, aby dowiedzieć się, ile
wierszy zostało przetworzonych przez instrukcję SQL.
Są dwa rodzaje kursorów:

- jawne
- niejawne

kursory niejawne są automatycznie definiowane przez ORACLE, dla pytań które zwracają jedynie jeden wiersz

kursory jawne są definiowane przez programistę dla pytań które zwracają więcej niż jeden wiersz

ATRYBUTY KURSORA DLA KURSORÓW NIEJAWNYCH:

- SQL%FOUND
- SQL%NOTFOUND
- SQL%ROWCOUNT

BAZODANOWE TRANSAKCJE

Wszystko musi zostać wykonane, aby transakcja powiodła się, inaczej całość transakcji jest cofana

ROZDZIAŁ 4

SŁOWNIK

CASE - Wyrażenie określające sposób działania na podstawie warunków, którego można używać poza blokiem PL/SQL w instrukcji SQL.
Condition - wyrażenie z wartościami prawda lub fałsz używane do podjęcia decyzji
IF - polecenie które pozwala na wybranie ścieżki w oparciu o warunek
LOOP - polecenie służące do powtarzania określonego kodu

IF STRUKTURA:

```
IF condition THEN
        statements;
ELSEIF condition THEN
        statements;
ELSE
        statements;
END IF
```

PORADNIK DO UŻYWANIA IF'ÓW:

- NULL działa w sumie podobnie do FALSE

CASE STRUKTURA:

```
variable_name :=
        CASE selector
                WHEN expression1 THEN result1
                WHEN expression2 THEN result2
                …
                WHEN expressionN THEN resultN
        END;
```

LOOP STRUKTURA:

```
BEGIN
        LOOP
                statements;
                EXIT [WHEN condition];
        END LOOP;
END;
```

WHILE STRUKTURA:

```
WHILE condition LOOP
        statement1;
        statement2;
        …
END LOOP;
```

FOR LOOP STRUKTURA:

```
FOR counter IN [REVERSE]
        lower_boud..upper_bound LOOP
        statement1;
        statement2;
END LOOP;
```

ROZDZIAŁ 5

Słownik

Active set - Zestaw wierszy zwrócony przez zapytanie wielowierszowe w jawnej operacji kursora
CLOSE -zwalnia obszar kontekstu i usuwa definicję aktywnego zestawu.
Context Area - zaalokowany obszar pamięci służący do przechowywania danych przetworzonych przez SQL'owe zapytanie
Cursor - Etykieta obszaru kontekstu lub wskaźnik do obszaru kontekstu
Explicit Cursor - zadeklarowany przez programistę do zadawania zapytań i zwracania więcej niż jednego wiersza
FETCH - polecenie, które zwraca aktualny wiersz i przesuwa kursor na następny wiersz do momentu, gdy wiersze się skończą, lub spełni się określony warunek
Implicit Cursor - zdefiniowany automatycznie przez ORACLE dla wszystkich poleceń DML
OPEN - polecenie, które wykonuje zapytanie powiązane z kursorem, identyfikuje aktywny zestaw i pozycję kursora na pierwszy wiersz
%ISOPEN - zwraca status kursora
%NOTFOUND - atrybut, który określa, czy pytanie może zwrócić jakieś wiersze w związku z zadanym zapytaniem
Record - złożone dane w PL/SQL, zawierające numery pól ze swoją nazwą i typem danych
%ROWCOUNT - atrybut, który przetwarza dokładną liczbę wierszy, lub zlicza ile wierszy zostało pobranych w pętli
%ROWTYPE - deklaruje rekord z takimi samymi polami jak kursor lub tabela na której bazuje
Cursor FOR loop - Automatyzuje standardowe operacje obsługi kursora, takie jak OPEN, FETCH, %NOTFOUND i CLOSE, dzięki czemu nie trzeba ich kodować jawnie
FOR UPDATE - deklaruje, że każdy wiersz jest zablokowany, dopóki jest pobierany, żeby inni użytkownicy nie mogli go zmodyfikować w trakcie gdy kursor jest otwarty
NOWAIT - słowo kluczowe używane do tego, żeby powiedzieć serwerowi oracle'a, żeby nie czekał, jeśli żądane wiersze są już zablokowane przez innego użytkownika

Kursory są po to, żeby móc pozyskać więcej niż jeden wiersz w zapytaniu.

Przykład jawnego kursora:

```
DECLARE
	CURSOR cur_depts IS
		SELECT department_id, department_name FROM departments;
	v_department_id departments.department_id%TYPE;
	v_department_name departments.department_name%TYPE;
BEGIN
	OPEN cur_depts;
	LOOP
		FETCH cur_depts INTO v_department_id, v_department_name;
		EXIT WHEN cur_depts%NOTFOUND;
		DBMS_OUTPUT.PUT_LINE();
	END LOOP;
	CLOSE cur_depts;
END;
```

Zamiast tworzyć 17 zmiennych do wydobycia całego wiersza z tabeli można po prostu użyć:

```
v_emp_record cur_emps%ROWTYPE;
```

%ISOPEN jest tak naprawdę do sprawdzenia czy kursor jest otwarty, żeby niepotrzebnie nie otwierać tego samego kursora ponownie, na przykład:

```
IF NOT cur_emps%ISOPEN THEN
	OPEN cur_emps
END IF;
LOOP
	FETCH cur_emps…
```

%ROWCOUNT jak chcemy warunek na pętli nałożyć, że na przykład tylko 10 wierszy chcemy pobrać, a %NOTFOUND jak chcemy całość, bądź gdy tabela będzie miała mniej niż ROWCOUNT to, żeby nie było błędu

Przykład FOR LOOP’a z kursorem:

```
FOR record_name IN cursor_name LOOP
	statement1;
	statement2;
```

```
END LOOP;
```

Realny przykład:

```
DECLARE
        CURSOR cur_emps IS
                SELECT employee_id, last_name FROM employees
                WHERE department_id = 50;
BEGIN
        FOR v_emp_record IN cur_emps LOOP
                DMBS_OUTPUT.PUT_LINE(v_emp_record.employee_id,
v_emp_record.employee.last_name);
        END LOOP;
END;
```

MOŻNA PÓŹNIEJ ROZBIĆ CAŁY REKORD NA POJEDYNCZE DANE Z TABELI

PORADNIK DO KURSORÓW:

- nie deklaruj rekordu do kontrolowania pętlą, bo dzieje się to niejawnie
- do tego rekordu można wtedy odwołać się jedynie w pętli. Poza pętlą nie ma takiej możliwości
- no i to co powyżej z tymi zmiennymi

KURSORY Z PARAMETRAMI

- oznacza to, że można otwierać kursor wielokrotnie z innym parametrem = inne dane

PRZYKŁAD:

```
DECLARE
        CURSOR cur_country (p_region_id NUMBER) IS
                SELECT country_id, country_name FROM countries
                WHERE region_id = p_region_id;
        v_country_record cur_country%ROWTYPE;
BEGIN
        OPEN cur_country (5);
        LOOP
                FETCH cur_country INTO v_country_record;
                EXIT WHEN cur_country%NOTFOUND;
                DBMS_OUTPUT.PUT_LINE();
        END LOOP;
        CLOSE cur_country;
END;
```

INNY PRZYKŁAD:

```
DECLARE
        v_dept_id employees.department_id%TYPE;
        CURSOR cur_emps (p_dept_id NUMBER) IS
                SELECT employee_id, salary FROM employees
                WHERE department_id = p_dept_id;
        v_emp_rec cur_emps%ROWTYPE;
BEGIN
        SELECT MAX(department_id) INTO v_dept_id FROM employees;
        OPEN cur_emps(v_dept_id)
        LOOP
                FETCH cur_emps INTO v_emp_rec;
                EXIT WHEN cur_emps%NOTFOUND;
                DBMS_OUTPUT.PUT_LINE();
        END LOOP;
        CLOSE cur_emps;
END;
```

KURSOR Z WIELOMA PARAMETRAMI:

```
DECLARE
        CURSOR cur_emps(p_job VARCHAR2, p_salary NUMBER) IS
                SELECT employee_id, last_name FROM employees
                WHERE job_id = p_job
                AND salary > p_salary;
BEGIN
        FOR v_emp_record IN cur_emps('IT_PROG', 10000) LOOP
        DBMS_OUTPUT.PUT_LINE();
        END LOOP;
END;
```

SYNTAX na NOWAIT i FOR UPDATE

```
CURSOR cursor_name IS
        SELECT … FROM …
        FOR UPDATE [OF column_reference] [NOWAIT | WAIT n];
```

PRZYKŁAD:

```
DECLARE
	CURSOR cur_emps IS
		SELECT employee_id, salary FROM employees
		WHERE salary <= 20000 FOR UPDATE NOWAIT;
	v_emp_rec cur_emps%ROWTYPE;
BEGIN
	OPEN cur_emps;
	LOOP
		FETCH cur_emps INTO v_emp_rec;
		EXIT WHEN cur_emps%NOTFOUND;
		UPDATE employees
			SET SALARY = v_emp_rec.salary*1.1
			WHERE CURRENT OF cur_emps;
	END LOOP;
	CLOSE cur_emps;
END;
```

PRZYKŁAD UŻYCIA DWÓCH KURSORÓW

```
DECLARE
	CURSOR cur_dept IS
		SELECT department_id, department_name
		FROM departments
		ORDER BY department_name;
	CURSOR cur_emp (p_dept_id NUMBER) IS
		SELECT first_name, last_name FROM employees
		WHERE department_id = p_dept_id;
		ORDER BY last_name;
	v_deptrec cur_dept%ROWTYPE
	v_emp_rec cur_emp%ROWTYPE
BEGIN
	OPEN cur_dept;
	LOOP
		FETCH cur_dept INTO v_deptrec;
		EXIT WHEN cur_dept%NOTFOUND;
		DBMS..();
		OPEN cur_emp;
		LOOP
			FETCH cur_emp INTO v_emp_rec;
			EXIT WHEN cur_emp%BOTFOUND
			DBMS…();
		END LOOP;
		CLOSE cur_emp;
	END LOOP;
	CLOSE cur_dept;
END;
```

PRZYKŁAD Z FOR

```
DECLARE
	CURSOR cur_dept IS SELECT * FROM departments;
	CURSOR cur_emp(p_dept_id NUMBER) IS
		SELECT * FROM employees
		WHERE department_id = p_dept_id
		FOR UPDATE NOWAIT;
BEGIN
	FOR v_dept_rec IN cur_dept LOOP
		DMBS..();
		FOR v_emp_rec IN cur_emp LOOP
			DMBS..();
			IF v_dept_rec.location_id = 1700 AND v_emp_rec.salary < 10000
				THEN UPDATE employees;
					SET salary = salary * 1.1
					WHERE CURRENT OF cur_emp;
			END IF;
```

```
            END LOOP;
        END LOOP;
END;
```

ROZDZIAŁ 6

SŁOWNIK

PL/SQL record - Złożony typ danych składający się z grupy powiązanych elementów danych przechowywanych jako pola, z których każde ma swoją własną nazwę i typ danych
Collection - nazwany zestaw wielokrotnych powtórzeń tego samego rodzaju danych
INDEX BY table - Kolekcja oparta na pojedynczym polu lub kolumnie
INDEX BY table of records – Kolekcja oparta na złożonym typie rekordów

USER DEFINED RECORDS
PRZYKŁAD:

```
TYPE type_name IS RECORD
        (field_declaration[,field_declaration]...);
identifier type_name;
```

```
DECLARE
        TYPE person_dept IS RECORD
        (first_name     employees.first_name%TYPE,
        last_name       employees.last_name%TYPE,
        department_name     departments.department_name%TYPE);
        v_person_dept_rec     person_dept;
BEGIN
        SELECT e.first_name, e.last_name, d.department_name
        INTO v_person_dept_rec
        FROM employees e JOIN department d
        ON e.department_id = d.department_id
        WHERE employee_id = 200;
        DBMS…();
END;
```

PRZYKŁAD 2:

```
DECLARE
        TYPE dept_info_type IS RECORD
        (department_id          departments.department_id%TYPE,
        department_name     department.department_name%TYPE);
        TYPE emp_dept_type IS RECORD
```

```
        (first_name     employees.first_name%TYPE,
        last_name       employees.last_name%TYPE
        dept_info       dept_info_type);

        v_emp_dept_rec      emp_dept_type;
BEGIN
        …
END;
```

INDEX BY TABLE SYNTAX

```
TYPE type_name IS TABLE OF DATA_TYPE
        INDEX BY PRIMARY_DATA_TYPE;
identifier type_name;
```

```
TYPE t_hire_date IS TABLE OF DATE
        INDEX BY BINARY_INTEGER;
v_hire_date_tab         t_hire_date;
```

POPULATING AN INDEX BY TABLE przykład:

```
DECLARE
        TYPE t_hire_date IS TABLE OF employees.hire_date%TYPE
        INDEX BY BINARY INTEGER;
        v_hire_date_tab         t_hire_date;
BEGIN
        FOR emp_rec IN
        (SELECT employee_id, hire_date FROM employees)
        LOOP
                v_hire_date_tab(emp_rec.employee_id) := emp_rec.hire_date;
        END LOOP;
END;
```

INDEX BY TABLE METHODS:

> EXISTS
- COUNT
- FIRST
- LAST

> PRIOR
- NEXT
- DELETE
- TRIM

INDEX BY TABLE OF records

EXAMPLE:

```
DECLARE
        TYPE t_emp_rec IS TABLE OF employees%ROWTYPE
        INDEX BY BINARY_INTEGER;
        v_emp_rec_tab         t_emp_rec;
BEGIN
        FOR emp_rec IN (SELECT * FROM employees) LOOP
        v_emp_rec_tab(emp_rec.employees_id) := emp_rec;
        DBMS_OUTPUT.PUT_LINE(v_emp_rec_tab(emp_rec.employee_id).salary);
        END LOOP:
END;
```

ROZDZIAŁ 7

SŁOWNIK

Exception - pojawia się, gdy błąd zostaje napotkany w trakcie pracy programu. Przerywa to działanie programu

Exception handler - kod, który definiuje akcje, które zostaną wykonane w przypadku, gdy kompilator napotka błąd w kodzie

Non-predefined Oracle Server Errors - każdy z nich ma swój kod błędu Oracle'a (ORA-nnnnn) i wiadomość, ale nie zdefiniowaną nazwę. Deklarujemy własne nazwy, żebyśmy mogli odwołać się do nich w sekcji EXCEPTION

Predefined Oracle Server Errors - każdy z nich ma swoją zdefiniowaną nazwę. Przykładowo jeżeli błąd ORA-01403 pojawi się, gdy nie pobierzemy żadnego wiersza z bazy w poleceniu SELECT, to PL/SQL rzuci tym predefiniowanym wyjątkiem: NO_DATA_FOUND

PRAGMA EXCEPTION_INIT - Polecenie, które nakazuje kompilatorowi skojarzenie nazwy wyjątku z numerem błędu Oracle. Pozwala to na odwołanie się do dowolnego wyjątku Oracle Server po nazwie i napisanie dla niego określonej procedury obsługi.

SQLERRM - zwraca dane znakowe zawierające wiadomość powiązaną z numerem błędu

SQLCODE - zwraca dane numeryczne dla kodu błędu

Raise - użyj tego polecenia do podniesienia nazwanego wyjątku

RAISE_APPLICATION_ERROR - procedura używana do zwrócenia zdefiniowanego przez użytkownika wyjątku kodu błędu i wiadomości z innego przechowywanego podprogramu

User-Defined error - nie są automatycznie podnoszone przez Oracle, ale są definiowane przez programistę i są umieszczone w kodzie

Exception Scope - część programu, gdzie wyjątki są zadeklarowane i dostępne

Exception Visibility - część programy gdzie wyjątki może być użyte (dostępne) bez używania kwalifikatora

Propagation of exceptions - wewnętrzny blok kończy się niepowodzeniem i PL/SQL przekazuje wyjątek do bloku zewnętrznego

RODZAJE:

- TOO_MANY_ROWS
- NO_DATA_FOUND
- INVALID_CURSOR
- ZERO_DIVIDE
- DUP_VAL_ON_INDEX
- WHEN OTHERS

JAK SIĘ DEFINIUJE BLOK EXCEPTION:

```
DECLARE
        v_country_name      countries.country_name%TYPE;
        v_elevation         countries.highest_elevation5TYPE;
BEGIN
        SELECT highest_elevation INTO v_elevation
        FROM countries WHERE country_name = v_country_name;
EXCEPTION
        WHEN NO_DATA_FOUND THEN
                DBMS_OUTPUT.PUT_LINE('tutaj komunikat gdy napotka się błąd');
END;
```

Wyjątek można deklarować w sekcji DECLARE

```
DECLARE
        e_insert_excep EXCEPTION;
        PRAGMA EXCEPTION_INIT(e_insert_excep, -01400);
BEGIN
        INSERT INTO departments
        (department_id, department_name)
        VALUES (280, NULL);
EXCEPTION
        WHEN e_insert_excep
THEN
        DBMS_OUTPUT.PUT_LINE('INSERT FAILED');
END;
```

w sekcji EXCEPTION można dodać SQLERRM i SQLCODE

```
EXCEPTION
        WHEN OTHERS THEN
                ROLLBACK;
        v_error_code := SQLCODE;
        v_error_message := SQLERRM;
INSERT INTO error_log(e_user, e_date, error_code, error_message)
VALUES(USER, SYSDATE, v_error_code, v_error_message);
```

```
END;
```

PRZYKŁAD TRAPPINGU EXCEPTIONA

```
DECLARE
        e_invalid_department EXCEPTION;
        v_name VARCHAR2(20) := ‘Accounting’;
        v_deptno NUMBER := 27;
BEGIN
        UPDATE departments
                SET department_name = v_name
                WHERE department_id = v_deptno;
        IF SQL%NOTFOUND THEN
                RAISE e_invalid_department;
        END IF;
EXCEPTION
        WHEN e_invalid_department THEN
                DBMS…();
END;
```

PRZYKŁAD RAISE_APPLICATION_ERROR

```
DECLARE
        v_mgr PLS_INTEGER := 123;
BEGIN
        DELETE FROM employees
        WHERE manager_id = v_mgr;
        IF SQL%NOTFOUND THEN
                RAISE_APPLICATION_ERROR(-20202, ‘This is not a valid manager’);
        END IF;
END;
```

Można to tak samo zdefiniować w sekcji EXCEPTION dla na przykład NO_DATA_FOUND itp

PRZYKŁAD użycia exceptiona w BLOKU WEWNĘTRZNYM

```
DECLARE
        v_last_name   employees.first_name%TYPE;
BEGIN
        BEGIN
                SELECT last_name INTO v_last_name
                FROM employees WHERE employee_id = 999;
                DBMS_OUT.PUT_LINE('Message 1');
        EXCEPTION
                WHEN TOO_MANY_ROWS THEN
                        DBMS_OUTPUT.PUT_LINE('Message 2');
        END;
        DBMS_OUTPUT.PUT_LINE('Message 3');
EXCEPTION
        WHEN OTHERS THEN
                DBMS_OUTPUT.PUT_LINE('Message 4');
END;
```

ROZDZIAŁ 8

SŁOWNIK

Anonymous Blocks - nienazwane, wykonywalne bloki PL/SQL, które nie mogą być ponownie użyte, gdybyśmy później chcieli z nich skorzystać
IS or AS - oznacza sekcję DECLARE podprogramu
Procedures - nazwane bloki PL/SQL, które mogą przyjmować parametry i są kompilowane i przechowywane w bazie danych
Subprograms - nazwane bloki PL/SQL, które są kompilowane i przechowywane w bazie danych
Argument - realna wartość przypisana parametrowi
Actual parameter - mogą to być wartości dosłowne, zmienne lub wyrażenia, które są podane na liście parametrów wywoływanego podprogramu.
Formal parameter - nazwa parametru zadeklarowana w nagłówku procedury
Parameters - przekazują i komunikują się z danymi pomiędzy programem wywołującym, a podprogramem i są powszechnie nazywane argumentami
Combination Notation - Wypisuje niektóre z rzeczywistych parametrów jako pozycyjne (bez operatora specjalnego) i niektóre jako nazwane (z operatorem =>).
IN Parameter - przekazuje wartość dla podprogramu to przetwarzania
IN OUT Parameter - zaopatrza wartość wejściową, która może zostać zwrócona jako zmodyfikowana wartość
Named Notation - Wypisuje wszystkie parametry we wcześniej ustalonej kolejności i używa operatorów przypisania (=>) to przypisania parametrowi odpowiadającej mu wartości
OUT Parameter - zwraca wartość do programu wywołującego
Postional Notation - wypisuje wszystkie parametry w takiej kolejności jak parametry formalne

SYNTAX NA TWORZENIE PROCEDUR

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[ ( parameter1 [mode1] datatype1,
   parameter2 [mode2] datatype2,
   …)] IS|AS
procedure_body;
```

EXAMPLE

```
CREATE OR REPLACE PROCEDURE add_dept IS
        v_dept_id       dept.departament_id%TYPE;
        v_dept_name  dept.department_name%TYPE;
BEGIN
        v_dept_id := 280;
        v_dept_name := 'ST-Curriculum';
        INSERT INTO dept(department_id, department_name)
        VALUES (v_dept_id, v_dept_name);
        DBMS_OUTPUT.PUT_LINE();
END;
```

Teraz można wywołać te procedurę

```
BEGIN
        add_dept;
END;
```

Można też stowrzyć procedurę w procedurze - lokalny podprogram
PRZYKŁAD:

```
CREATE OR REPLACE PROCEDURE delete_emp (p_emp_id IN
employees.employee_id%TYPE)
IS
        PROCEDURE log_emp (p_emp IN employees.employee_id%TYPE)
        IS BEGIN
                INSERT INTO logging_table VALUES (p_emp…);
        END log_emp;
BEGIN
        DELETE FROM employees
                WHERE employee_id = p_emp_id;
        log_emp(p_emp_id);
END delete_emp;
```

TWORZENIE PROCEDURY Z PARAMETRAMI:

```
CREATE OR REPLACE PROCEDURE raise_salary
(p_id IN employees.employee_id%TYPE,
 p_percent IN NUMBER)
IS
BEGIN
        UPDATE employees
                SET salary = salary * (1 + p_percent/100)
                WHERE employee_id = p_id;
END raise_salary;
```

wywołanie:

```
BEGIN
        raise_salary(176,10);
END;
```

PRZYKŁAD UŻYCIA OUT PARAMETRÓW

```
CREATE OR REPLACE PROCEDURE query_emp
(p_id IN        employees.employee_id%TYPE,
 p_name OUT employees.first_name%TYPE,
 p_salary OUT employees.salary%TYPE)
IS
BEGIN
        SELECT first_name, salary INTO p_name, p_salary
        FROM employees
        WHERE employee_id = p_id;
END query_emp;
```

WYWOŁANIE

```
DECLARE
        a_emp_name  employees.first_name%TYPE;
        a_emp_salary employees.salary%TYPE;
BEGIN
        query_emp(178, a_emp_name, a_emp_salary);
END;
```

SYNTAX DLA PRZEKAZYWANIA PARAMETRÓW

- Pozycyjny
- nazwany
- kombinacyjny

EXAMPLE:

```
CREATE OR REPLACE PROCEDURE add_dept(
        p_name IN departments.department_name%TYPE,
        p_loc IN      departments.location_id%TYPE)
IS
BEGIN
        INSERT INTO department(department_id, department_name, location_id) VALUES
        (departments_seq.NEXTVAL, p_name, p_loc);
END add_dept
```

PRZEKAZANIE POZYCYJNE:

```
add_dept('EDUCATION', 1400);
```

PRZEKAZANIE NAZWANE:

```
add_dept(p_loc=>1400, p_name=>'EDUCATION');
```

PRZEKAZANIE KOMBINACYJNE:

```
add_dept('EDUCATION', p_loc=>1400);
```

W PRZYPADKU PRZEKAZYWANIA KOMBINACYJNEGO, ZAWSZE JAKO PIERWSZY MUSI IŚĆ POZYCYJNY PARAMETR, PÓŹNIEJ NAZWANY

jest też opcja DEFAULT dla parametrów IN

wtedy ma on wartość podstawową i nie trzeba się martwić, że się nie przekaże mu wartości np.

```
CREATE OR REPLACE PROCEDURE add_dept(
        p_name departments.department_name%TYPE := 'Unknown',
        p_loc     departments.location_id%TYPE := 1400)
IS
BEGIN
….
```

ROZDZIAŁ 9

SŁOWNIK

Funkcje składowane – nazwany blok PL/SQL, który może akceptować opcjonalne parametry IN i musi zwracać pojedynczy wynik.
Funkcja zdefiniowana przez użytkownika – funkcja utworzona przez programistę PL/SQL, której można używać
wszędzie tam, gdzie występuje wartość lub funkcja
ALL_* tables - zawiera informacje o obiektach do których ma się dostęp
Data Dictionary - katalog wszystkie obiektów bazy danych zawartej w bazie danych oracle
DBA_* tables - zawiera informacje o wszystkim w bazie danych, nie ważne kto jest ich właścicielem
USER_* tables - zawiera informacje o obiektach, których jesteś właścicielem
ALL_SOURCE - tabela która zawiera kod źródłowy podprogramów, których właścicielami są wszyscy inni, którzy dali Ci uprawnienia do wykonywania ich
USER_OBJECTS - tabela zawierająca informacje o nazwach i typach procedur i funkcji
USER_SOURCE - tabela zawierająca kody źródłowe wszystkich podprogramów, których jesteś właścicielem
ALTER privilege - umożliwia wybranej osobie, której damy pozwolenie na zmianę tabeli
EXECUTE privilege - umożliwia użytkownikowi do wywoływania i wykonywania PL/SQL podprogramu
INDEX privilege - pozwala wybranej osobie, której damy uprawnienia do tworzenia indeksów w tabeli
Object privilege - pozwala do używania określonych obiektów bazy danych, takich jak tabele, widoki, procedury przez więcej niż jednego użytkownika bazy
REFERENCES privilege - pozwala wybranej osobie, której damy uprawnienia do sprawdzania istnienia wierszy w tabeli przy użyciu kluczy obcych
Autonomous Transaction - podprogramy, które pozwalają sesji mieć dwie lub więcej aktywnych transakcji w tym samym czasie
Definer's Rights - podprogram wykonujący się w sesji użytkownika, ale z przywilejami i nazwami tabeli osoby, która jest ich twórcą
Invoker's Rights - podprogram wykonujący się w sesji użytkownika, z przywilejami użytkownika, kluczami i synonimami

SYNTAX DLA TWORZENIA FUNKCJI

```
CREATE [OR REPLACE] FUNCTION function_name
        [(parameter1 [mode1] datatype 1, …)]
RETURN datatype IS|AS
        [local_variable_declarations; …]
BEGIN
        – actions
        RETURN expression
END [function_name];
```

PRZYKŁAD

```
CREATE OR REPLACE FUNCTION get_sal(
        p_id IN employees.employee_id%TYPE)
        RETURN NUMBER IS
        v_sal employees.salary%TYPE :=0;
BEGIN
        SELECT salary
        INTO v_sal
        FROM employees
        WHERE employee_id = p_idl
        RETURN v_sal;
END get_sal;
```

Wywołać można w taki sposób:

```
DECLARE
   v_sal employees.salary%TYPE;
BEGIN
   v_sal := get_sal(100);
   DBMS_OUTPUT.PUT_LINE(v_sal);
END;
```

i w taki:

```
DBMS_OUTPUT.PUT_LINE(get_sal(100));
```

teraz można to wykorzystać w innych podprogramach itp:

Przykład:

```
CREATE OR REPLACE FUNCTION valid_dept (p_dept_no
departments.department_id%TYPE)
RETURN BOOLEAN IS v_valid VARCHAR2(1);
BEGIN
        SELECT 'x‘ INTO v_valid FROM departments
        WHERE department_id = p_dept_no;
        RETURN(true);
EXCEPTION WHEN NO_DATA_FOUND THEN
        RETURN(false);
WHEN OTHERS THEN NULL;
END;
```

Wywołanie:

```
DECLARE
   v_istnieje BOOLEAN;
BEGIN
   v_istnieje := valid_dept(10); -- Wywołanie funkcji dla ID 10

   IF v_istnieje THEN
      DBMS_OUTPUT.PUT_LINE('Departament istnieje.');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Departament nie istnieje.');
   END IF;
END;
```

RÓŻNICE MIĘDZY FUNKCJĄ A PROCEDURĄ:

Procedura:

- Wykonywana jako instrukcja PL/SQL
- nie zawiera klauzuli RETURN w nagłówku
- może zwrócić wartości jako parametry wyjściowe
- Może zawierać klauzulę RETURN bez wartości

Funkcja:

- Wykonywana jako część wyrażenia
- musi zawierać klauzulę RETURN w nagłówku
- musi zwrócić pojedynczą wartość
- musi zawierać przynajmniej jedną klauzulę RETURN

DO TWORZENIA OPERACJI DML LEPIEJ UŻYWAĆ PROCEDURY > FUNKCJE

Aby zobaczyć wszystkie tabele w słowniku, należy użyć:
DESCRIBE ALL_TABLES

można też zawsze sprawdzić więcej rzeczy, na przykład właściciela tabeli:
SELECT table_name, owner FROM ALL_TABLES;

Aby zobaczyć wszystkie obiekty, których się jest właścicielem to:
SELECT object_type, object_name FROM USER_OBJECTS;

ABY POZBYĆ SIĘ PROCEDURY BĄDŹ FUNKCJI:
DROP PROCEDURE/FUNCTION procedure_name

NADAWANIE UPRAWNIEŃ

GRANT object_priv [(columns)]
ON object
TO {user|role|PUBLIC}
[WITH GRANT OPTION];

np.

GRANT INSERT, UPDATE
    ON employees TO TOM, SUSAN;

GRANT SELECT
    ON departments TO PUBLIC;

ABY ODEBRAĆ UPRAWNIENIA

REVOKE object_priv [(columns)]
ON object
FROM {user|role|PUBLIC};

np.

REVOKE INSERT, UPDATE ON employees FROM TOM, SUSAN

REVOKE SELECT ON departments FROM PUBLIC;

można też dać uprawnienia na funkcji czy procedurze, np.

```
CREATE OR REPLACE PROCEDURE add_dept…
IS BEGIN
…
        INSERT INTO DEPARTMENTS … ;
END;

GRANT EXECUTE ON add_dept TO SUSAN;
```

ROZDZIAŁ 10

SŁOWNIK

Encapsulation - Dwuczęściowa struktura, w której szczegółowy kod treści pakietu jest niewidoczny dla środowiska wywołującego, które widzi jedynie specyfikację. Jeśli konieczne są zmiany w kodzie, treść pakietu można edytować i rekompilować bez konieczności edycji lub rekompilacji specyfikacji.
OR REPLACE - Opcja usuwająca i ponownie tworząca treść pakietu
Package body - zawiera wykonywalną część kodu podprogramów, które zostały zadeklarowane w specyfikacji pakietu. Może też zawierać swoje własne deklaracje zmiennych
Package Specification - Interfejs aplikacji, który deklaruje konstrukcje (procedury, funkcje, zmienne itd.) widoczne dla środowiska wywołującego.
PL/SQL packages - Kontenery umożliwiające grupowanie powiązanych podprogramów PL/SQL, zmiennych, kursorów i wyjątków
Private components - zadeklarowane tylko w ciele pakietu i można się do nich odwoływać tylko przez inne (publiczne lub prywatne) konstruktory w tym samym pakiecie
Public components -są deklarowane w specyfikacji pakietu i mogą być wywoływane z dowolnego środowiska wywołującego, pod warunkiem, że użytkownikowi przyznano uprawnienie EXECUTE dla pakietu.
Visibility - Opisuje, czy komponent jest widoczny, czyli czy można do niego odwoływać się i czy jest używany przez inne komponenty lub obiekty.

Package specification:

- deklarowane jest jako pierwsze
- deklaruje procedury, funkcje, zmienne i inne, które są widoczne dla środowiska wywołującego

Package Body:

- zawiera wykonywalną część kodu podprogramu, który został zadeklarowany w package specification
- może zawierać swoje własne zadeklarowane zmienne

Syntax dla tworzenia specyfikacji pakietu:

```
CREATE [OR REPLACE] PACKAGE package_name
IS/AS
        public type and variable declarations
        public subprogram specifications
END [package_name];
```

Przykład check_emp_pkg:

```
CRATE OR REPLACE PACKAGE check_emp_pkg
IS
        g_max_length_of_service      CONSTANT NUMBER := 100
        PROCEDURE chk_hiredate
                (p_date IN employees.hire_date%TYPE);
        PROCEDURE chk_dept_mgr
                (p_emp_id      IN      employees.employee_id%TYPE,
                p_mgr          IN      employees.manager_id%TYPE);
END check_emp_pkg;
```

Syntax do tworzenia body pakietu:

```
CREATE [OR REPLACE] PACKAGE BODY package_name IS|AS
        private type and variable declarations
        subprogram bodies
[BEGIN initialization statements]
END [package_name];
```

Przykład do check_emp_pkg

```
CREATE OR REPLACE PACKAGE BODY check_emp_pkg IS
        PROCEDURE chk_hiredate
                (p_date         IN      employees.hire_date%TYPE)
                IS BEGIN
                        IF MONTHS_BETWEEN(SYSDATE, p_date) >
                        g_max_length_of_service * 12 THEN
                        RAISE_APPLICATION_ERROR(-20200, ‘Invalid Hiredate’);
                        END IF
        END chk_hiredate;
        PROCEDURE chk_dept_mgr
                (p_emp_id       IN      employees.employee_id%TYPE,
                p_mgr           IN      employees.manager_id%TYPE)
                IS BEGIN…
        END chk_hiredate;
END check_emp_pkg;
```

można łatwo zobaczyć co pakiet zawiera za pomocą:

DESCRIBE check_emp_pkg

- procedury
- argumenty (czy są IN czy OUT) i ich Datatype

Dlaczego używać pakietów:

- Modularność - powiązane programy i zmienne mogą być zgrupowane razem
- Chowanie informacji - tylko deklaracje w specyfikacji pakietu są widoczne dla wywołujących
- Twórcy aplikacji nie muszą znać szczegółów kodu z body
- łatwiejsza konserwacja - można zmienić i rekompilować kod z body pakietu bez potrzeby kompilowania specyfikacji
- Dlatego aplikacje, które już korzystają z pakietu, nie muszą być ponownie kompilowane.

WAŻNE:

- publiczne komponenty są deklarowane w specyfikacji
- prywatne komponenty są deklarowane w body

w obrębie pakietu są trzy miejsca w których można zadeklarować komponenty:

- globalnie w specyfikacji - widać je w body pakietu przez środowisko wywołujące
- lokalnie w body - widoczne w obrębie body pakietu, ale nie przez środowisko wywołujące
- lokalnie w body pakietu w obrębie określonego podprogramu - widoczne tylko w tym podprogramie

żeby usunąć pakiet to wystarczy:

```
DROP PACKAGE package_name;
```

Aby zobaczyć pakiety w dictionary wystarczy:

```
(specyfikacja)
SELECT text
        FROM user_source
        WHERE name = 'SALARY_PKG' AND type = 'PACKAGE'
        ORDER BY line;

(body)
SELECT text
        FROM user_source
        WHERE name = 'SALARY_PKG' AND type = 'PACKAGE_BODY'
        ORDER BY line;
```

jak na przykład mamy błędy przy kompilacji pakietu to można je zobaczyć w USER_ERRORS;

```
SELECT line, text, position
        FROM USER_ERRORS
        WHERE NAME = 'BAD_PROC' AND type = 'PROCEDURE'
        ORDER BY sequence;
```

Overloading:

- tworzenie dwóch lub więcej pakietów z tą samą nazwą, wewnątrz tego samego pakietu
- pozwala na budowanie elastycznych rozwiązań wywoływania przeciążonych podprogramów na podstawie argumentów podawanych podczas wywołania przeciążonego podprogramu CHAR vs NUMBER vs DATE
- łatwiejsze dla programisty, bo wystarczy, że zapamięta jedną nazwę podprogramu
- Overloading może być wykonywany za pomocą podprogramów w pakiecie, ale nie w przypadku samodzielnych podprogramów

przykładowo

```
CREATE OR REPLACE PACKAGE emp_pkg IS
        PROCEDURE find_emp
                (p_employee_id IN NUMBER, p_last_name OUT VARCHAR2);
        PROCEDURE find_emp
                (p_job_id IN  VARCHAR2, p_last_name OUT VARCHAR2);
        PROCEDURE find_emp
                (p_hiredate IN DATE, p_last_name OUT VARCHAR2);
END emp_pkg;
```

restrykcje overloadingu, nie możesz overloadować:

- Dwa podprogramy, jeśli ich parametry formalne różnią się jedynie typem danych, a różne typy danych należą do tej samej kategorii (NUMBER i INTEGER należą do tej samej kategorii; VARCHAR2 i CHAR należą do tej samej kategorii)
- Dwie funkcje różnią się jedynie zwróconym typem, nawet jeśli są w innych kategoriach
- jeśli nazwy parametrów są te same

ROZDZIAŁ 11

SŁOWNIK

- **Package state** - kolekcja zmiennych pakietu i ich aktualnych wartości w sesji
- **Persistence** - odnosi się do cech obiektu i procesu, które istnieją nawet po zakończeniu procesu, który je stworzył
- **DBMS_OUTPUT package** - pakiet, które wysyła tekstowe wiadomości z jakiegokolwiek bloku PL/SQL do prywatnego miejsca w pamięci, z którego wiadomość może być wyświetlona na ekranie
- **UTL_FILE package** - pakiet, który pozwala programom PL/SQL odczytywać i pisać pliki tekstowe
- **UTL_MAIL package** - pakiet który pozwala programom PL/SQL do zarządzania komponentami wiadomości e-mail

Package state jest:

- inicjalizowany kiedy pakiet jest pierwszy raz ładowany
- stały (domyślnie) podczas życia sesji
- przechowywany w prywatnej pamięci sesji
- unikalny dla każdej sesji, nawet jeśli druga sesja zacznie się przez tego samego użytkownika

do UTL_FILE potrzeba:

- directory
- filename
- open_mode

do UTL_MAIL potrzeba:

- sender
- recipients
- message
- subject

Więcej do poczytania samemu bo nudy i ciężko pisać wszystko

ROZDZIAŁ 12

SŁOWNIK:

Native Dynamic SQL – niektóre programy muszą budować i przetwarzać różnorodne instrukcje SQL w czasie wykonywania. Takie instrukcje mogą, i prawdopodobnie będą, zmieniać się w trakcie wykonywania.
EXECUTE IMMEDIATE – instrukcja, która przygotowuje (analizuje) i natychmiast wykonuje dynamiczną instrukcję SQL lub anonimowy blok PL/SQL.
Bulk Binding - wyciąga wszystkie wiersze w pojedynczym wywołaniu do SQL Engine
BULK COLLECT Clause - zapewnia przetwarzanie zbiorcze dla SELECT i FETCH
DETERMINISTIC Clause – oznacza, że ta sama wartość wejściowa zawsze generuje tę samą wartość wyjściową i musi być użyta do utworzenia indeksu opartego na funkcjach we własnych funkcjach.
FORALL – zapewnia przetwarzanie zbiorcze dla aktywności DML
NOCOPY hint – przekazuje argumenty przez referencję, a nie przez wartość i zazwyczaj przyspiesza wykonywanie instrukcji SQL.
RETURNING Clause – umożliwia pobieranie danych zmodyfikowanych przez instrukcję DML bez wywoływania oddzielnego przełączenia kontekstu

Wszystkie programy w bazie danych przechodzą przez różne etapy:

- Parse (pre-execution): czy to możliwe? sprawdza syntax, czy obiekty istnieją, przywileje
- Bing: pobieranie rzeczywistych wartości dla zmiennych, które są w programie
- Execute: wykonywanie programu
- Fetch: wyniki zwracane do użytkownika

Dynamic SQL:
− Jest konstruowany i przechowywany jako ciąg znaków w podprogramie
− Jest instrukcją SQL ze zmiennymi danymi kolumnowymi lub różnymi warunkami z symbolami zastępczymi (zmiennymi wiążącymi) lub bez nich
− Umożliwia pisanie i wykonywanie instrukcji definicji danych, kontroli danych lub kontroli sesji z poziomu PL/SQL

Native Dynamic SQL:

- PL/SQL nie wspiera DDL statementów pisanych bezpośrednio w programie
- NDS pozwala obejść ten problem poprzez konstruowanie i przechowywanie kodu SQL jako ciągu znaków w podprogramie.
- NDS jest wykonywany wraz z EXECUTE IMMEDIATE lub DBMS_SQL pakietem
- pozwala na wykonywanie procesów SQL, których struktura nie jest jeszcze znana
- może używać też OPEN-FOR, FETCH i CLOSE

syntax:

```
EXECUTE IMMEDIATE dynamic_string
[ INTO {define_variable
        [, define_variable ] … | record}]
[USING [IN|OUT|IN OUT] bind_argument
        [, [IN|OUT\IN OUT] bind_argument] … ];
```

przykład:

```
CREATE PROCEDURE drop_any_table(p_table_name VARCHAR2) IS
BEGIN
        EXECUTE IMMEDIATE 'DROP TABLE ' || p_table_name;
END;
CREATE PROCEDURE drop_any_table(p_table_name VARCHAR2) IS
        v_dynamic_stmt        VARCHAR2(50);
BEGIN
        v_dynamic_stmt := 'DROP TABLE ' || p_table_name;
        EXECUTE IMMEDIATE v_dynamic_stmt;
END;

BEGIN
        drop_any_table('EMPLOYEE_NAMES');
END;
```

procedury i funkcje pakietu DMBS_SQL:

- OPEN_CURSOR
- PARSE
- BIND_VARIABLE
- EXECUTE
- FETCH_ROWS
- CLOSE_CURSOR

do przeczytania 11 i 12 rozdział, bo nudy jak sam skurwysyn

ROZDZIAŁ 13

SŁOWNIK

Application trigger - wykonują się automatycznie kiedy dana sytuacja występuje w obrębie aplikacji
Database trigger - wykonuje się automatycznie kiedy jakieś zdarzenie (takie jak DML lub DDL) lub system event jka (logowanie, wyłączenie) pojawia się w bazie danych
Triggers - programy do wykonywania specyficznych akcji, kiedy dany event następuje w aplikacji bądź w bazie danych
DML trigger - trigger, który jest automatycznie wykonywany kiedy jakieś polecenie DML (INSERT, UPDATE, DELETE) jest wykonywany
Row trigger - wykonuje się raz dla każdego wiersza objętego w dany event

Statement trigger - jest wykonywany raz w imieniu zdarzenia wyzwalającego, niezależnie od tego ile wierszy (lub czy w ogóle) jest objętych danym zdarzeniem
Conditional Predicate - predefiniowane zmienne boolowskie INSERTING, DELETING i UPDATING, które mogą być testowane w body triggera, aby kod wybrał odpowiednią drogę w zależności który DML wywołał trigger do działania
Compound Trigger - pojedynczy trigger, który zawiera akcje dla czterech z możliwych timing points: przed poleceniem triggera, przed każdym wierszem, po każdym wierszy i po poleceniu triggera
DML Row Trigger - trigger DML, który wykonuje się tylko raz dla każdego wiersza objętego zdarzeniem DML
INSTEAD OF trigger - trigger, który zastępuje polecenie DML, na złożony widok z poleceniami DL na tablicy, na której widoku bazuje
:OLD i :NEW qualifiers - Umożliwiają row triggerrowi do dostępu do wartości kolumn w wierszu tabeli, który jest aktualnie modyfikowany przez polecenie triggera
Database event triggers - są wywoływane przez nie-SQL zdarzenia w bazie danych
DDL Triggers - są wywoływane przez polecenia DDL: CREATE, ALTER lub DROP
Mutating table - tabela, która jest obecnie modyfikowana przez UPDATE, DELETE lub INSERT, lub tabela która może wymagać aktualizacji w wyniku deklaratywnej akcji integralności referencyjnej DELETE CASCADE
ALL_TRIGGERS - szczegółowy kod i status wszystkich triggerów w bazie do, których użytkownik ma dostęp
USER_TRIGGERS - szczegółowy kod i status triggerów, których właścicielem jest obecny użytkownik

Przykład triggera:

```
CREATE OR REPLACE TRIGGER log_sal_change_trigg
AFTER UPDATE OF salary ON employees
BEGIN
        INSERT INTO log_table(user_id, logon_date) VALUES (USER,SYSDATE);
END;
```

Trigger to:

- blok PL/SQL powiązany z określonym zdarzeniem, na przykład logowaniem przez użytkownika
- wykonuje się automatycznie kiedy dochodzi do danego zdarzenia
- przechowywany w bazie danych

RÓŻNICA Między Triggerem, a procedurą/funkcją jest taka, że procedury bądź funkcje muszą być wywołane **jawnie**

Database triggers:

- automatycznie sie wykonują gdy zdarzenie tj. DML czy DDL lub jakieś inne zdarzenie wystąpi w bazie
- tworzone i przechowywane w bd, tak jak procedury i funkcje

Application Triggers:

- wykonują się kiedy dane zdarzenie będzie miało miejsce wewnątrz aplikacji
- nie są częścią bazy danych

Triggery mogą być:

- row-level - odpala się raz, dla każdego wiersza dotkniętego przez polecenie triggera
- statement-level - odpala się raz dla całej instrukcji

Możliwe użycia dla triggerów:

- zwiększenie reguł bezpieczeństwa dla dużej bazy danych
- tworzenie jakichś raportów automatycznie
- egzekwuje złożone więzy integralności
- tworzy raporty logowania automatycznie
- chronią tabele przez przypadkowym usunięciem
- chronią przed nieprawidłowymi transakcjami DML

Poradnik do triggerów:

- nie definiuj triggera do duplikowania bądź zastępowania akcji, które można łatwo zrobić innymi sposobami
- nadużywanie triggerów może prowadzić do wolnego przetwarzania złożonych zależności, co może być trudne w utrzymaniu
- używaj triggerów tylko kiedy to konieczne i pamiętaj o rekursji (trigger może wywoływać sam siebie) i efektów kaskadowych
- unikaj długiej logiki triggerów przez tworzenie procedur bądź pakietów które znajduje się w ciele triggera

Trigger:

- CREATE TRIGGER
- source code w USER_TRIGGERS
- niejawnie wywoływany
- COMMIT, SAVEPOINT i ROLLBACK nie są dozwolone

Procedure:

- CREATE PROCEDURE
- source code w USER_SOURCE
- jawnie wywoływany
- COMMIT, SAVEPOINT i ROLLBACK dozwolone

Syntax do DML triggera:

```
CREATE [OR REPLACE] TRIGGER trigger_name
        timing
        event1 [OR event2 OR event3] ON object_name
trigger_body
```

przykładowo

```
CREATE OR REPLACE TRIGGER sal_upd_trigg
BEFORE UPDATE OF salary ON employees
BEGIN … END;

CREATE OR REPLACE TRIGGER emp_del_trigg
AFTER DELETE ON employees
BEGIN … END;
```

mamy opcje do tego triggera:

- BEFORE:
- AFTER:
- INSTEAD OF:

może też mieć wiele opcji:

```
CREATE OR REPLACE TRIGGER emp_del_trigg
AFTER INSERT OR DELETE OR UPDATE ON employees
BEGIN … END;
```

Ten trigger:

- odpala się tylko raz dla każdego wykonania się polecenia od triggera (nawet jeśli żaden wiersz nie został dotknięty zmianą)
- jest domyślnym typem DML triggera
- odpala się raz nawet jeśli żaden wiersz nie został dotknięty zmianą

np.

```
CREATE OR REPLACE TRIGGER log_emp_changes
AFTER UPDATE ON employees BEGIN
        INSERT INTO log_emp_table (who, when) VALUES (USER, SYSDATE);
END;
```

Przykład z predicamentami: DELETING, INSERTING, UPDATING

```
CREATE OR REPLACE TRIGGER secure_emp
        BEFORE INSERT OR UPDATE OR DELETE ON employees
BEGIN
        IF TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN') THEN
        IF DELETING THEN RAISE_APPLICATION_ERROR(-20501 '...');
        ELSIF INSERTING THEN RAISE_APPLICATION_ERROR(-20502 '...');
        ELSIF UPDATING THEN RAISE_APPLICATION_ERROR(-20503 '...);
        END IF;
        END IF;
END;
```

można też na przykład ograniczyć się do jednej kolumny:

```
IF UPDATING('SALARY') THEN …
ELSIF UPDATING('JOB_ID') THEN …
```

ROW TRIGGER wystarczy dodać informację FOR EACH ROW, i wtedy już wiadomo, że mamy do czynienia z Row Triggerem:

```
CREATE OR REPLACE TRIGGER log_emps
        AFTER UPDATE OF salary ON employees FOR EACH ROW
BEGIN…
```

:OLD odnosi się do starej wartości w kolumnie, jeżeli jesteśmy w trakcie wykonywania na niej operacji (jakiś update)

przykładowo:

```
CREATE OR REPLACE TRIGGER log_emp
        AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
        INSERT INTO log_emp_table (who, when, which_employee, old_salary, new_salary)
        VALUES (USER, SYSDATE, :OLD.employee_id, :OLD.salary, :NEW,salary);
END;
```

Używanie REFERENCING clause:

```
CREATE OR REPLACE TRIGGER log_emps
        AFTER UPDATE OF salary ON old FOR EACH ROW
```

old znaczy:

- jest to kwalifikator (tak jak :NEW)
- i też jest to nazwa tabeli

bo tam dalej w kodzie jest użycie :OLD. i to jest confusing
ale nie trzeba używać ani OLD, ani NEW, można użyć Referencing clause'a

PRZYKŁAD:

```
CREATE OR REPLACE TRIGGER log_emps
        AFTER UPDATE OF salary ON old
REFERENCING OLD as former NEW as latter
FOR EACH ROW
BEGIN
        INSERT INTO log_emp_table (who, when, which_employee, old_salary, new_salary)
        VALUES (USER, SYSDATE, :former.employee_id, :former.salary, :latter.salary);
END;
```

Można też dodać taki warunek:

```
CREATE OR REPLACE TRIGGER log_emps
        AFTER UPDATE OF salary ON employees
FOR EACH ROW
WHEN (NEW.salary > OLD.salary)
```

INSTEAD OF TRIGGERS

```
CREATE OR REPLACE TRIGGER new_emp_dept
        INSTEAD OF INSERT ON emp_details
BEGIN
        INSERT INTO new_emps
        VALUES (:NEW.employee_id, :NEW.last_name, :NEW.salary, :NEW.department_id);
        UPDATE new depts
        SET dept_sal = dept_sal + :NEW.salary
        WHERE department_id = :NEW.department_id;
END;
```

COMPOUND TRIGGER:

- pojedynczy trigger, który zawiera akcje dla każdego z czterech możliwych timingów
- ma sekcje deklaracji i sekcje dla każdego z timing pointsów
- nie trzeba wszystkich używać, tylko te które chcemy

przykład:

```
CREATE OR REPLACE TRIGGER log_emps
        FOR UPDATE OF salary ON copy_employees
COMPOUND TRIGGER
TYPE t_log_emp IS TABLE OF log_table%ROWTYPE
INDEX BY BINARY_INTEGER;
log_emp_tab t_log_emp;
v_index BINARY_INTEGER := 0;
AFTER EACH ROW IS
BEGIN
        v_index := v_index + 1;
        log_emp_tab(v_index).employee_id := :OLD.employee_id;
        log_emp_tab(v_index).change_date := SYSDATE;
        log_emp_tab(v_index).salary := :NEW.salary;
END AFTER EACH ROW;
AFTER STATEMENT IS
BEGIN
        FORALL I IN log_emp_tab.FIRST..log_emp_tab.LAST
        INSERT INTO log_table VALUES log_emp_tab(i);
END AFTER STATEMENT;
END log_emps;
```

SYNTAX dla triggerów DDL

```
CREATE [OR REPLACE] TRIGGER trigger_name
Timing
[ddl_event1 [OR ddl_event2 OR …]]
ON {DATABASE|SCHEMA}
trigger_body
```

EXAMPLE

```
CREATE OR REPLACE TRIGGER log_create_trigg
AFTER CREATE ON SCHEMA
BEGIN
        INSERT INTO log_table VALUES (USER,SYSDATE);
END;
```

one mogą być tylko na **DATABASE albo SCHEMA**, nie na konkretnej tabeli i **nie można używać** :OLD ani :NEW

drugi EXAMPLE

```
CREATE OR REPLACE TRIGGER prevent_drop_trigg
BEFORE DROP ON SCHEMA
BEGIN
        RAISE_APPLICATION_ERROR(-20203, 'Attempted drop - failed');
END;
```

Poradnik do tworzenia triggerów na bazie danych

- nie można używać INSTEAD OF z database event triggers
- można je zdefiniować jako odpowiedź na logowanie, wyjście i nawet SERVERERROR
- ON SCHEMA nie można używać z SHUTDOWN i STARTUP eventami

LOGON I LOGOFF triggers:

```
CREATE REPLACE TRIGGER logon_trig
AFTER LOGON ON SCHEMA
BEGIN
        INSERT INTO log_trig_table(user_id, log_date, action_
        VALUES (USER, SYSDATE, 'Logging on');
END;
```

```
CREATE OR REPLACE TRIGGER logoff_trig
BEFORE LOGOFF ON SCHEMA
BEGIN
        INSERT INTO log_trig_table(user_id, log_date, action)
        VALUES (USER, SYSDATE, 'Logging off');
END;
```

można w triggerze wywołać też procedurę jakąś przechowywaną w bazie

Mutating table i row triggery:

- mutating table to tabela która aktualnie jest poddawana zmianie (jest mutowana)
- row trigger nie może użyć SELECT z mutującej tabeli ponieważ widziałby niezgodny zestaw danych (dane w tabeli zmieniały by się, w tym samym czasie co trigger by chciał je odczytać)
- jednakże row trigger może użyć SELECT z innej tabeli jeśli to konieczne
- nie odnoszą się te restrykcje do poleceń DML triggera, tylko do DML row triggera

aby uniknąc mutating table errors:

- row-level trigger nie może pytać ani modyfikować mutującej tabeli
- statement-level trigger nei może pytań ani modyfikować mutującej tabeli jeżeli trigger zostanie odpalony w wyniku usunięcia CASCADE
- czytanie i wpisywanie danych używając triggerów podlega pewnym zasadom
- restrykcje podlegają tylko row triggerom, chyba, że polecenie triggera zostanie uruchomione jako wynik ON DELETE CASCADE

EXAMPLE OF MUTATING TABLE:

```
CREATE OR REPLACE TRIGGER check_salary
        BEFORE INSERT OR UPDATE OF salary, job_id ON employees
FOR EACH ROW
DECLARE
        v_minsalary employees.salary%TYPE
        v_maxsalary employees.salary%TYPE
BEGIN
        SELECT MIN(salary), MAX(salary)
        INTO v_minsalary, v_maxsalary
        FROM employees
        WHERE job_id = :NEW.job_id;
                IF :NEW.salary < v_minsalary OR
                        :NEW.salary > v_maxsalary THEN
                        RAISE_APPLICATION_ERROR(-20505, '...');
                END IF;
END;
```

POMYSŁY NA UŻYCIE TRIGGERÓW:

pomysł nr 1

```
CREATE OR REPLACE TRIGGER weekdays_emp
        BEFORE UPDATE ON employees
BEGIN
        IF (TO_CHAR (SYSDATE, 'DY) IN ('SAT', 'SUN')) THEN
                RAISE_APPLICATION_ERROR(-20506, '...');
        END IF;
END;
```

nie można zmieniać tabeli employees w weekend, nikt nie może

Przykład drugi na triggera, zamiast constraina na minmalne wynagrodzenie:

```
CREATE OR REPLACE TRIGGER check_salary
        BEFORE UPDATE ON salary ON employees
FOR EACH ROW
        WHEN (NEW.salary < OLD.salary OR NEW.salary < 500)
BEGIN
        RAISE_APPLICATION_ERROR(-20508, '...');
END;
```

trzeci przykład na raporcik

```
CREATE OR REPLACE PROCEDURE increment_salary
        (p_id IN NUMBER, p_new_sal IN NUMBER) IS
BEGIN
        UPDATE copy_departments
        SET total_salary = total_salary + NVL(p_new_sal, 0)
        WHERE department_id = p_id
END increment_salary;

CREATE OR REPLACE TRIGGER compute_salary
AFTER INSERT OR UPDATE OF salary OR DELETE ON employees
FOR EACH ROW
BEGIN
        IF DELETING THEN increment_salary
                (:OLD.department_id, (:OLD.salary * -1));
        ELSIF UPDATING THEN increment_salary
                (:NEW.department_id, (:NEW.salary - :OLD.salary));
        ELSE increment_salary
                (:NEW.department_id, :NEW.salary);
        END IF;
END;
```

Przywileje potrzebne dla triggerów:

Aby tworzyć na własnym schemacie to potrzeba:

- CREATE trigger przywilej systemowy
- zwykłe przywileje (SELECT, UPDATE, EXECUTE… itd.) na innych obiektach w schemacie do których się odnosimy w body triggera
- ALTER privilege na tabeli lub widoku powiązanym z triggerem

jak zobaczyć triggera w data dictionary?

- USER_OBJECTS - nazwy i typy obiektów
- USER_TRIGGERS - szczegółowy kod i status triggerów
- USER_ERRORS - PL/SQL syntax errors (błędy kompilacji) triggerów
- Kod źródłowy triggerów jest w USER_TRIGGERS, a nie USER_SOURCE

można zmieniać status triggerów (wyłączyć tymczasowo czy coś)

ALTER TRIGGER trigger_name DISABLE|ENABLE

ALTER TABLE table_name DISABLE | ENABLE ALL TRIGGERS;

rekompilacja triggera dla tabeli:

ALTER TRIGGER trigger_name COMPILE;

usuwanie triggerów:

DROP TRIGGER trigger_name

ROZDZIAŁ 14

SŁOWNIK

Remote Dependencies - Dzieje się tak, gdy baza danych Oracle zarządza zależnościami w rozproszonym środowisku w sieci.
Signature Mode - to ustawienie służące do sprawdzania czy lokalny podprogram jest nadal prawidłowy, czy też musi być unieważniony poprzez sprawdzenie wartości - unikalnej numeru obliczanego i przechowywanego za każdym razem gdy procedura jest rekompilowana
Timestamp Mode - to ustawienie służące do sprawdzania czy lokalny podprogram jest nadal prawidłowy, czy musi być unieważniony poprzez sprawdzenie wartości w widoku słownika USER_OBJECTS

Problemy zależności:

- jeżeli modyfikujesz coś w obiekcie referencyjnym (podrzędnym), obiekt który korzysta z jego zależności, może przestać pracować poprawnie
- przykładowo jeżeli zmieni się struktura tabeli, to procedura może nie działać poprawnie
- serwer Oracle automatycznie zapisuje zależności wśród obiektów
- aby zobaczyć zależności, wszystkie schematy mają status (valid lub invalid) który jest zapisany w DATA DICTIONARY i można to sprawdzić poprzez wyciągnięcie statusu z USER_OBJECTS

lokalne zależności:

- lokalne zależności to takie, gdy obiekty są na tym samym węźle w tej samej bazie danych
- wszystkie lokalne są przetwarzania automatycznie przez serwer oracle'a w tabeli "depends-on"
- jeżeli jakiś obiekt zależny jest zmodyfikowany, to ten "rodzic" jest ustawiany jako invalidated
- kiedy wywołujemy obiekt który jest invalidated to oracle automatycznie próbuje go kompilować
- jeżeli ten obiekt jest w innej bazie danych to ta zależność nazywana jest **remote dependency**
- dwie różne tabele mają osobne data dictionaries, to niektóre rzeczy działają inaczej dla remote dependencies

zobaczenie direct dependencies:

```
SELECT name, type, referenced_name, referenced_type
    FROM user_dependencies
    WHERE referenced_name IN ('EMPLOYEES', 'EMP_VW');
```

Rekompilacja może być automatyczna poprzez uruchomienie programu lub:

1. ALTER PROCEDURE [SCHEMA.]procedure_name COMPILE
2. ALTER FUNCTION [SCHEMA.]function_name COMPILE
3. ALTER PACKAGE [SCHEMA.]package_name COMPILE [PACKAGE | SPECIFICATION | BODY];
4. ALTER TRIGGER trigger_name [COMPILE[DEBUG]];

rekompilacja się nie uda gdy:

- obiekt zależny został usunięty lub jego nazwa została zmieniona
- typ danych w kolumnie zależnej się zmienił
- zależna kolumna została usunięta
- zależny widok został zmieniony z innym widokiem z innymi kolumnami
- lista parametrów zależnej procedury została zmodyfikowana

rekompilacja zależnych procedur i funkcji będzie poprawna gdy:

- nowe kolumny zostaną dodane do referencyjnej tabeli
- wszystkie polecenia INSERT zawierają listę kolumn
- żadna nowa kolumna nie została zdefiniowana jako NOT NULL
- typ danych referencyjnych kolumn nie zmienił się
- prywatna tabela została usunięta, ale publiczna tabela z tą samą nazwą i strukturą istnieje
- ciało PL/SQL referencyjnej procedury zostało zmodyfikowane i skompilowane z sukcesem

Jak zmniejszyć ryzyko dependency failures:

- deklarowanie rekordów z atrybutem %ROWTYPE
- deklarowanie zmiennych z atrybutem %TYPE
- zadawanie zapytań z notacją SELECT *
- Dołączanie listy kolumn za pomocą instrukcji INSERT

REMOTE DEPENDENCIES

- gdy obiekty do których się odwołujemy i obiekty zależne są w dwóch innych bazach danych, to zależność nazywa się remote dependency
- oracle server nie śledzi automatycznie relacji (takich jak zależności) pomiędzy obiektami w dwóch innych bazach danych, nawet jeśli te bazy są na tym samym serwerze
- to dlatego, że każda baza ma swój osobny niezależny DATA DICTIONARY

jako pierwsze baza albo programista tworzy database link, który jest wskaźnikiem do remote database i dopiero później referencję do tej bazy w swojej lokalnej procedurze, np.

```
CREATE DATABASE LINK my_db_link
        CONNECT TO remote_username IDENTIFIED BY remote_password
        USING ‘remote_database_name’;
        CREAT EOR REPLACE procedureA…
IS BEGIN
        procedureB@my_db_link (parameters…) … END;
```

Zrozumienie remote dependecies:

- jeżeli remote procedure jest (directly lub indirectly) invalidated, server nie może automatycznie unieważnić procedury lokalnej, ponieważ w remote dicitionary nie ma żadnego odniesienia do tej lokalnej procedury
- jak są w tej samej bazie to może dać invalidate, ale jak nie ma ich to nie
- ale może zostać invalidate remotely, gdy się ją wywołuje w czasie egzekwowania jej

są dwie możliwości na zarządzanie zdalnymi zależnościami:

- TIMESTAMP
- SIGNATURE

żeby używać SIGNATURE to trzeba odpalić:
REMOTE_DEPENDENCIES_MODE = [SIGNATURE | TIMESTAMP]

TIMESTAMP:

- kiedy obiekt w bazie jest tworzony lub modyfikowany (na przykład gdy procedura jest egzekwowana) oracle autoamtycznie zapisuje timestamp change do słownika
- w USER_OBJECTS można je zobaczyć

```
SELECT object_name, timestamp
    FROM USER_OBJECTS
    WHERE object_type = 'PROCEDURE';
```

Gdy lokalna procedura A jest kompilowana i odwołuje się do
zdalnej procedury B, znacznik czasu zdalnej procedury jest odczytywany i zapisywany w lokalnej procedurze, podobnie jak znacznik czasu lokalnej procedury.

Przykład

jeśli procedura A zostanie skompilowana o 7:00, to kompilator sprawdza validity i timestamp tej zdalnej procedury B, poprzez network call i przechowuje timestamp B w słowniku A,
Czyli w słowniku A jest timestamp z A i B
W słowniku B jest tylko timestamp B

Signature mode:

- unikalny numer zwany podpisem jest obliczany i zapisywany za każdym razem gdy procedura jest rekompilowana
- jest kalkulowane na podstawie: nazwy procedury, typów danych parametrów, trybów parametrów (IN/OUT/IN OUT);
- Czyli signature mode zmienia się tylko gdy nazwa procedury lub parametry ulegną zmianie. W body może się zmieniać wszystko

Przykład:

```
Remote at 8:00 a.m.>
CREATE OR REPLACE FUNCTION remote_func
RETURN NUMBER IS
        v_remote_count NUMBER;
BEGIN SELECT COUNT(*) INTO v_remote_count FROM employees;
        RETURN v_remote_count;
END remote_func;

Local at 9:00 a.m.>
CREATE OR REPLACE PROCEDURE local_proc IS
        v_local_count NUMBER;
BEGIN
        v_local_count:= remote_func@remote_database_name;
END local_proc;
```

ROZDZIAŁ 15
SŁOWNIK

PLSQL_CODE_TYPE - poprawia prędkość wykonywania się skompilowanego kodu PL/SQL
PLSQL_OPTIMIZE_LEVEL - używane do kontrolowania co PL/SQL kompilator zrobi z bezużytecznym kodem, daje benefity w postaci lepszego performance'u
PLSQL INITIALIZATION PARAMETER - używany do zmiany sposobu w jaki sesja bazy użytkownika wchodzi w interakcje z serwerem Oracle
USER_PLSQL_OBJECT_SETTINGS - widok data dictionary, w którym można zobaczyć jak program został skompilowany
DBMS_WARNING - Pakiet dostarczany przez serwer, który pozwala ustawić te same kategorie ostrzeżeń co parametr PLSQL_WARNINGS, ale też pozwala na zapisanie poprzednich ustawień błędów w zmiennych, do których można wrócić i ustawić ponownie jako defaultowe
PL/SQL Compiler errors - fatalny błąd zidentyfikowany podczas kompilacji, które muszą zostać naprawione, aby program mógł się poprawnie skompilować
PL/SQL Compiler warnings - nie-fatalna sytuacja, która może zostać zidentyfikowana podczas kompilacji przez ustawienia PLSQL_WARNINGS. Używanie WARNINGS, pozwala na selektywne wybieranie, które kategorie błędów wyłączyć, włączyć lub traktować jako błędy
Conditional Compilation - Umożliwia dołączenie kodu źródłowego do kodu PL/SQL, który może zostać skompilowany lub zignorowany przez kompilator w zależności od wartości sprawdzanej podczas kompilacji
DBMS_DB_VESRION - pakiet określa numery wersji Oracle i inne przydatne do wyboru prostej kompilacji warunkowej na podstawie wersji Oracle'a
DBMS_PREPROCESSOR - Pakiet dostarczany przez Oracle'a, który może zwrócić lub wyświetlić przetworzony tekst źródłowy zapisanej jednostki PL/SQL
Inquiry and Selection Directives - używane w kompilacji warunkowej

PLSQL_CCFLAGS - mechanizm pozwalający programistom PL/SQL kontrolować kompilację warunkową każdej biblioteki i niezależnej jednostki
USER_SOURCE - widok DATA DICTIONARY, w którym można zobaczyć kompletny kod źródłowy, zawierający dyrektywy kompilatora
DBMS_DDL.CREATE_WRAPPED - procedura zaciemniania (obfuscation) pojedynczej jednostki PL/SQL, która zarówno opakowuje tekst i tworzy jednostkę PL/SQL
Obfuscation - proces przemiany kodu w kod maszynowy, aby zrobić go trudnego do zrozumienia
Wrapper Utility - przetwarza plik wejściowy SQL i zaciemnia tylko jednostki PL/SQL w pliku
Wrapping PL/SQL Source Code - proces ukrywania kodu źródłowego PL/SQL, za pomocą użycia wrap utility lub podprogramów DBMS_DDL

Parametry inicjalizacji:

- są używane do zmiany sposobu interakcji sesji z serwerem Oracle
- wszystkie parametry inicjalizacji mają nazwę, typ danych i domyślną wartość
- mogą być używane do regulowania ochrony, zwiększenia wydajności i wielu innych
- wiele z nich nie ma nic wspólnego z PL/SQL

Dwa parametry:

- PLSQL_CODE_TYPE - jest to varchar2 z możliwymi wartości INTERPRETED (domyślna wartość) i NATIVE
- PLSQL_OPTIMIZE_LEVEL - jest to numer z możliwymi wartościami 0, 1, 2 (domyślna) i 3

Przykład użycia PLSQL_CODE_TYPE

wpisujemy przez odpaleniem programu:

ALTER SESSION SET PLSQL_CODE_TYPE = NATIVE;
i wykonuje się z zawrotną prędkoścIa.
a
ALTER SESSION SET PLSQL_CODE_TYPE = INTERPRETED
i jest o 10 sekund wolniej

```
CREATE OR REPLACE PROCEDURE longproc IS
        v_number PLS_INTEGER;
BEGIN
        FOR i IN 1..5000000 LOOP
                v_number := v_number * 2;
                v_number := v_number  / 2;
        END LOOP;
END;
```

NATIVE COMPILATION:

- tworzy natywny kod PL/SQL, ale nie natywny kod SQL (nie ma czegoś takiego!)
- dlatego silnik PL/SQL wykonuje to szybciej, ale polecenia SQL w takiej samej prędkości
- i ogólnie SQL statements wykonują się wolniej w szczególności, gdy tabela zawiera tysiące wierszy
- aby to przyspieszyć to jest ten Bulk Binding i indeksy

PLSQL_OPTIMIZE_LEVEL:

- może być używania do kontroli co kompilator zrobi z bezużytecznym kodem, co daje lepszą wydajność
- jest wartość jest od 0 - 3
- im wyższa wartość tym więcej wysiłku kompilator potrzebuje do zoptymalizowania kodu do wykonania
- zarówno to daje plusy dla natively compiled i interpreted compilation.
- domyslnie przyjmuje wartość 2

Jak dasz 0, to będzie wolno, ale będzie działać ze starszymi oprogramowaniami oracle'a
Jak dasz 1 to kompilator usunie niepotrzebny kod i wyjątki
Jak dasz 2 to kompilator usunie bezużyteczny kod tak jak w 1, ale też czasem przesunie kod w inne miejscu, żeby wykonał się szybciej
Jak dasz 3 to dostaniesz te benefity co powyżej + subprogram inlining. Znaczy to, że kompilowany kod innego podprogramu zostanie skopiowany do wywoływanego podprogramu, co w wyniku da tylko jedną jednostkę do wykonania

aby włączyć warning messages należy wpisać ALTER SESJE

ALTER SESSION SET PLSQL_WARNINGS = 'ENABLE:ALL';

wtedy będzie informować o ostrzeżeniach

kategorie PL/SQL warnings:

- SEVERE - kod, który może wywołać niespodziewane zachowanie lub zły wynik
- PERFORMANCE - kod który może spowodować wolne wykonanie programu
- INFORMATIONAL - inne słabe techniki kodowania (kod ktory nigdy sie nie wykona)

jak chcesz pojedyncze to przykładowo:

ALTER SESSION SET PLSQL_WARNINGS = 'ENABLE:PERFORMANCE';
albo dodać dwa, idk jak kto woli

DBMS_WARNING
- można zmienić lub ustawić kategorie ostrzeżeń za pomocą tego
- można dzięki temu te ustawienia wjebać w zmienną do łatwego ich zapamiętania

DBMS_WARNING:
- SET_* - nowe ustawienia zastępują stare (całość)
- ADD_* - tylko zmiana wybranych ustawień
- GET_* - nie zmienia, są przechowywane w zmiennej

Conditional Compilation:
- pozwala na przechowywanie kodu źródłowego w programie PL/SQL, który może zostać skompilowany, bądź zignorowany. Zignorowany na podstawie:
    - wartości parametru inicjalizacyjnego
    - wersji oprogramowania oracle
    - wartości stałej globalnego pakietu
    - innych warunków, które sobie chcesz ustawić
- kontrolujesz to za pomocą dolara albo dwóch ($ or $$)

na przykład:

```
CREATE OR REPLACE PROCEDURE lets_pretend IS
BEGIN
        …
        $IF MS_OFFICE_VERSION >= ‘2018’ $THEN
                include_holographics
        $ELSE
                include_std_graphic
        $END
END lets_pretend;
```

PRINT_POST_PROCESSED_SOURCE:
- procedura zwracająca kod źródłowy tekstu przechowywanego w jednostce PL/SQL, która jest wynikiem ostatniej udanej kompilacji jej kodu źródłowego
- podstawowe parametryL
    - object_type - PACKAGE, PACKAGE_BODY, PROCEDURE, FUNCTION or TRIGGER
    - schema_name - to jest nazwa użytkownika
    - object_name - to nazwa obiektu

PLSQL_CCFLAGS:
- pozwala na ustawienie wartości dla zmiennych i testowanie tych zmiennych w programie PL/SQL
- definiujesz i nadajesz wartości tym zmiennym używając PLSQL_CCFLAGS
- każda zmienna PLSQL_CCFLAGS musi być boolean lub PLS_INTEGER
- testujesz wartości tych zmiennych za pomocą dyrektyw zapytania
- używasz do tego double dolar sign ($$)

```
ALTER SESSION SET PLSQL_CCFLAGS = 'debug:true';
CREATE OR REPLACE PROCEDURE testproc IS
BEGIN
        ...
        $IF $$debug $THEN
                DBMS_OUTPUT.PUT_LINE('This code was executed');
        $END
        ...
END testproc;
```

Obfuscating:

- kod źródłowy lub maszynowy który jest trudny do zrozumienia
- celowo programiści to robią, aby zakryć cel kodu
- programy (obsfucators) zamienają czytelny kod na taki zaciemniony używając określonych technik
- robi się to aby zarządzać ryzykiem (niepożądane osoby nie znały kodu źródłowego)

Są dwa sposoby na zaciemnienie:

- DBMS_DDL.CREATE_WRAPPED - pakiet oracle'a
- PL/SQL wrapper utility program, WRAP

DMBS_DDL_PACKAGE zawiera procedury do zaciemnienia jednostek, takich jak specyfikacje pakietów, body, funkcji, procedur itp. Zawiera funkcję WRAP

przykład:

```
BEGIN
        DBMS_DDL.CREATE_WRAPPED
        ('CREATE OR REPLACE PROCEDURE mycleverproc
                (p_param1 IN NUMBER, p_param2 OUT NUMBER)
        IS BEGIN
                kod
        END mycleverproc;');
END;
```

można też to samo zrobić z pojedynczymi linijkami

```
DECLARE
        v_code VARCHAR2(32767);
BEGIN
        v_code:= 'CREATE OR REPLACE FUNCTION myclevererfunc '
        || '(p_param1 IN NUMBER) ' || 'RETURN NUMBER IS BEGIN '
        || '... /* some even cleverer but private code
        */ ' || '
        END myclevererfunc;';
DBMS_DDL.CREATE_WRAPPED(v_code);
END;
```

Wrapper utility 3 kroki:

- stworzenie pliku tekstowego zawierający cały unwrapped source_code
- wykonanie WRAP do stworzenia drugiego pliku zawierajacego wrapped code
- połączenie z bazą danych i wykonanie wrapped text file jako skryptu do skompilowania wrapped codu do Data dictionary

```
C:> WRAP INAME=mysourcecode.sql
```

aby dodać drugi

```
C:> WRAP INAME=mysourcecode.sql ONAME=mywrappedcode.plb
```

porównanie obu metod:

- dla bardzo dużych programu musisz użyć WRAP utility
- dla małych DBMS_DDL.CREATE_WRAPPED jest łatwiejsze i nie trzeba się logować na serwerze bazy